

B3 – COORDINATION AGILE EN EQUIPE

Campus Companion —Formation B3

Table des matières

1. Objet de l'annexe	1
2. Contexte : une équipe de 5, une contrainte de 48 heures.....	1
3. Organisation Agile retenue	2
3.1 Découpage en lots autonomes.....	2
3.2 Un contrat d'interface stable comme mécanisme de découplage	2
3.3 Intégration par petits incréments	2
3.4 Intégration continue via pull requests	2
4. Mon rôle dans la coordination	3
5. Bilan et limites.....	3

1. Objet de l'annexe

Cette annexe documente la compétence C2.9 — « Coordination Agile de la programmation en équipe » — à travers le projet Campus Companion Ynov, réalisé en équipe de 5 développeurs lors d'un Challenge de 48 heures (30-31 mars 2026).

Contrairement aux Annexes B1 et B2, aucun outil de gestion Agile dédié (board Kanban, backlog Trello ou Notion) n'a été utilisé pendant ce challenge : la coordination s'est faite de façon pragmatique, par la structuration du dépôt en lots fonctionnels indépendants et par des échanges oraux directs entre les membres de l'équipe, adaptés à la contrainte extrême de temps. Cette annexe s'appuie donc uniquement sur des preuves vérifiables dans l'historique Git du dépôt (branches, pull requests, fréquence et granularité des commits), complétées par une analyse de ce que ce mode de fonctionnement a permis — et de ses limites.

2. Contexte : une équipe de 5, une contrainte de 48 heures

Le projet a réuni 5 contributeurs : julien-gournay, Inkflow59, Simon-Pruvot, aZeDen31 et mohammed-tahri24. Il fallait livrer, en 48 heures, un assistant conversationnel décliné en trois clients (mobile Flutter, web React, desktop Electron) reposant sur un unique backend Spring Boot. Un délai aussi court interdisait toute cérémonie Agile lourde (sprint planning formel, rituel quotidien chronométré, rétrospective) : l'équipe a dû choisir les pratiques Agile strictement utiles à la coordination, en écartant celles dont le coût en temps aurait dépassé le bénéfice sur un format aussi court.

Lot fonctionnel	Branche Git	Pull request de fusion
Backoffice (desktop Electron)	backoffice	PR #1 « Implémentation du backoffice », PR #2 « FAQ : ajout d'une boîte de dialogue... »

Lot fonctionnel	Branche Git	Pull request de fusion
Mobile (Flutter)	mobile	PR #3 « Merge de la branche "mobile" »
Web (React)	web	PR #4 « Merge de la branche web »
Backend (Spring Boot)	dépôt séparé ycompagnon-api	développé et versionné indépendamment

3. Organisation Agile retenue

3.1 Découpage en lots autonomes

Le premier mécanisme de coordination a été la répartition du travail en lots fonctionnels étanches (mobile / web / desktop / backend), chacun développé sur sa propre branche. Ce découpage a permis à chaque membre de l'équipe d'avancer en parallèle sans attendre les autres, une condition indispensable pour tenir le délai de 48 heures.

3.2 Un contrat d'interface stable comme mécanisme de découplage

Ce découpage en lots parallèles n'aurait pas fonctionné sans un point de synchronisation commun : le contrat d'API (endpoints, formats JSON, authentification), documenté et stabilisé tôt dans le projet (cf. Annexes B1 et B2). En figeant ce contrat en amont, l'équipe a pu développer les trois clients sans dépendance bloquante vis-à-vis de l'avancement du backend — chaque client pouvait être testé et itéré dès lors que le contrat était respecté, même si l'implémentation backend continuait d'évoluer.

3.3 Intégration par petits incréments

L'historique de commits du dépôt montre une cadence de livraison fine plutôt que de gros blocs de code livrés en une fois : au moins 27 commits fonctionnels (hors fusions) répartis sur les 30 et 31 mars 2026, avec des messages ciblés sur une unité de travail précise :

- « Ajout du header, et du formulaire qui ne s'affiche que s'il y a une erreur dans le tchat »
- « Rendu le formulaire responsive »
- « Ajout des noms des input »
- « Ajout de la suppression des conversations et des confirmations »
- « FAQ : ajout d'une boîte de dialogue de modification et d'un aperçu des réponses »

Cette granularité correspond au principe Agile de livraison incrémentale : chaque commit reste petit, testable et réversible, ce qui limite le risque de conflit de fusion et permet d'intégrer les branches fréquemment plutôt qu'en une seule fois en fin de challenge.

3.4 Intégration continue via pull requests

Les quatre lots ont été réintégrés dans la branche principale via quatre pull requests dédiées (une par lot, avec une itération supplémentaire pour le backoffice). Ce mécanisme, même informel, a joué le rôle d'un point de contrôle avant fusion, évitant qu'une branche ne diverge trop longtemps de la version principale — pratique directement héritée de l'intégration continue promue par les méthodes Agile.

4. Mon rôle dans la coordination

J'ai développé le client web (branche web) ainsi que la couche d'intégration à l'API (classe CampusCompanionAPI, cf. Annexe B2), et contribué à la documentation du contrat d'API utilisé par l'ensemble de l'équipe (README.md, API_DOCUMENTATION.md, cf. Annexe B1). En stabilisant ce contrat tôt et en le documentant, j'ai fourni à mes coéquipiers un point de repère commun leur permettant de développer leurs clients respectifs sans avoir à se synchroniser en continu avec moi sur les détails d'implémentation du backend — une forme de coordination Agile par contrat plutôt que par supervision directe.

5. Bilan et limites

Ce mode de coordination, pragmatique et adapté à la contrainte de 48 heures, a permis de livrer les trois clients et le backend dans les temps. Il présente cependant une limite assumée : l'absence d'outil de suivi visuel (Kanban, backlog) rend le partage des tâches restant et l'avancement de chacun moins traçables a posteriori que dans un cadre Scrum classique. Sur un projet de plus longue durée, cette même équipe gagnerait à formaliser un board de tâches partagées, même léger, pour fiabiliser le suivi au-delà du seul historique Git.